Python Engineering - Class 10 Theory Guide

Advanced Functions

This guide explains the programming, mathematical and/or engineering theory behind the exercise. The goal is to understand the model before memorizing syntax.

1. Function signatures

A signature defines how information is allowed to enter a function.

Relationship / equation

```
f(positional, default=value, *args, **kwargs)
```

Connection to code

Choosing a clear signature is an interface-design decision.

2. Packing

*args and **kwargs collect an unknown number of incoming arguments into standard Python structures.

Relationship / equation

```
*args -> tuple; **kwargs -> dict
```

Connection to code

This adds flexibility but should not replace clear explicit parameters when the interface is known.

3. Unpacking

The star operators at the call site do the opposite: they expand an existing sequence or mapping into individual arguments.

Relationship / equation

```
f(*values); f(**config)
```

Connection to code

For **config, dictionary keys must match parameter names expected by the function.

4. Functions as data

A function can be stored, passed and selected before it is called.

Relationship / equation

```
operation = add vs result = add(5,3)
```

Connection to code

This idea later powers the TESTS list in the custom test runner.

5. Higher-order behavior

A function such as apply_operation does not know the exact operation; it receives the behavior as an argument.

Relationship / equation

```
apply_operation(operation,x,y)
```

Connection to code

This separates orchestration from the mathematical operation itself.

Manual validation method

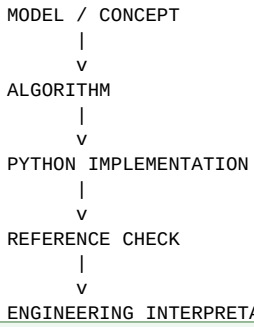
For any exercise in this class, use the following protocol before trusting the output:

- 1. Write input values with units or types.
- 2. Predict sign, scale, type and shape of the result.
- 3. Manually calculate a small case when possible.
- 4. Run the code and compare against the prediction.
- 5. Change one input in a controlled way and explain the effect.

Connection to the rest of the course

Flexible functions and function objects prepare callbacks, solvers and the custom test runner.

Final summary



Syntax is the tool. The transferable skill is being able to model, structure, validate and interpret.